

“Travel time reliability on highway using Bluetooth/ WiFi sensors”

**Indian Institute of Technology Jammu
Electrical department**

TABLE OF CONTENTS

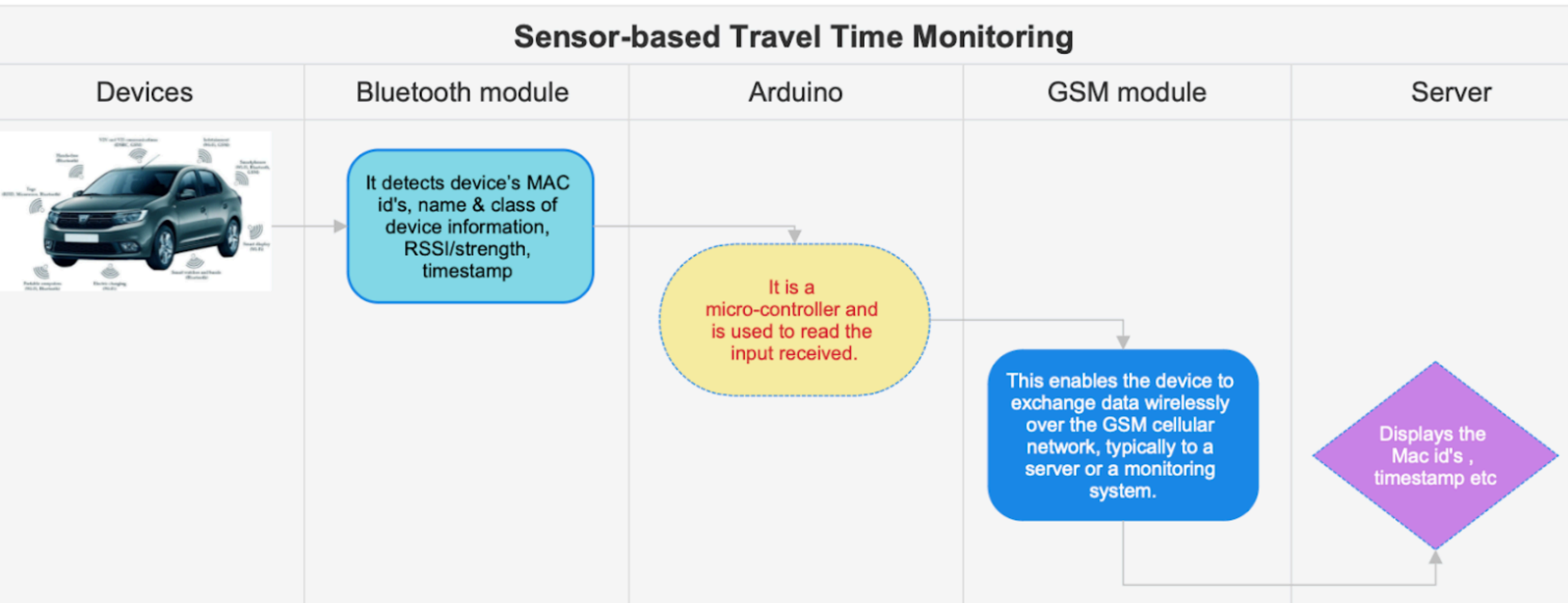
- **Objectives**
- **Streamline diagram**
- **Hardware and Software requirements**
- **Methodology**
- **Our Product**
- **Advantages**
- **Progress**
- **References**

OBJECTIVE

- **Develop a device to improve travel time reliability on highways by accurately measuring and analyzing traffic conditions.**
- **The project aims to detect Bluetooth and WiFi signals from vehicles, enabling the collection of real-time traffic data for analysis and provide valuable insights into congestion levels, and travel times.**

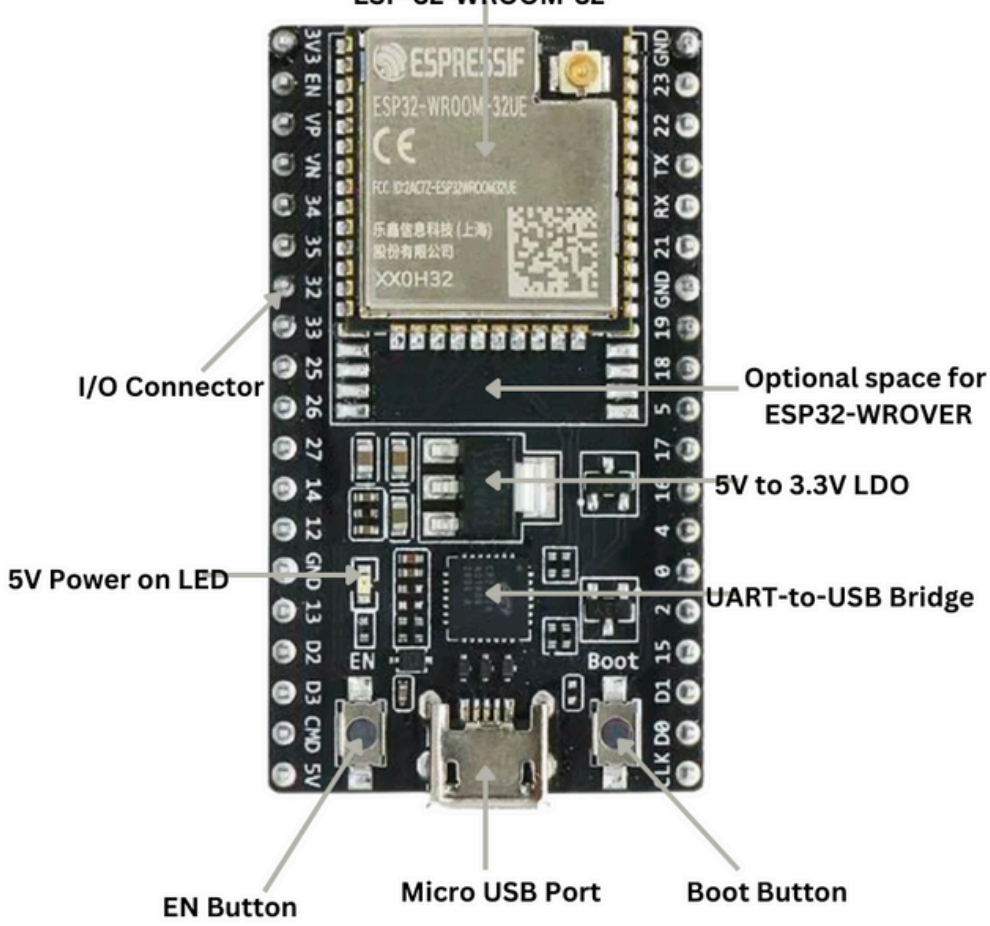


Streamline diagram

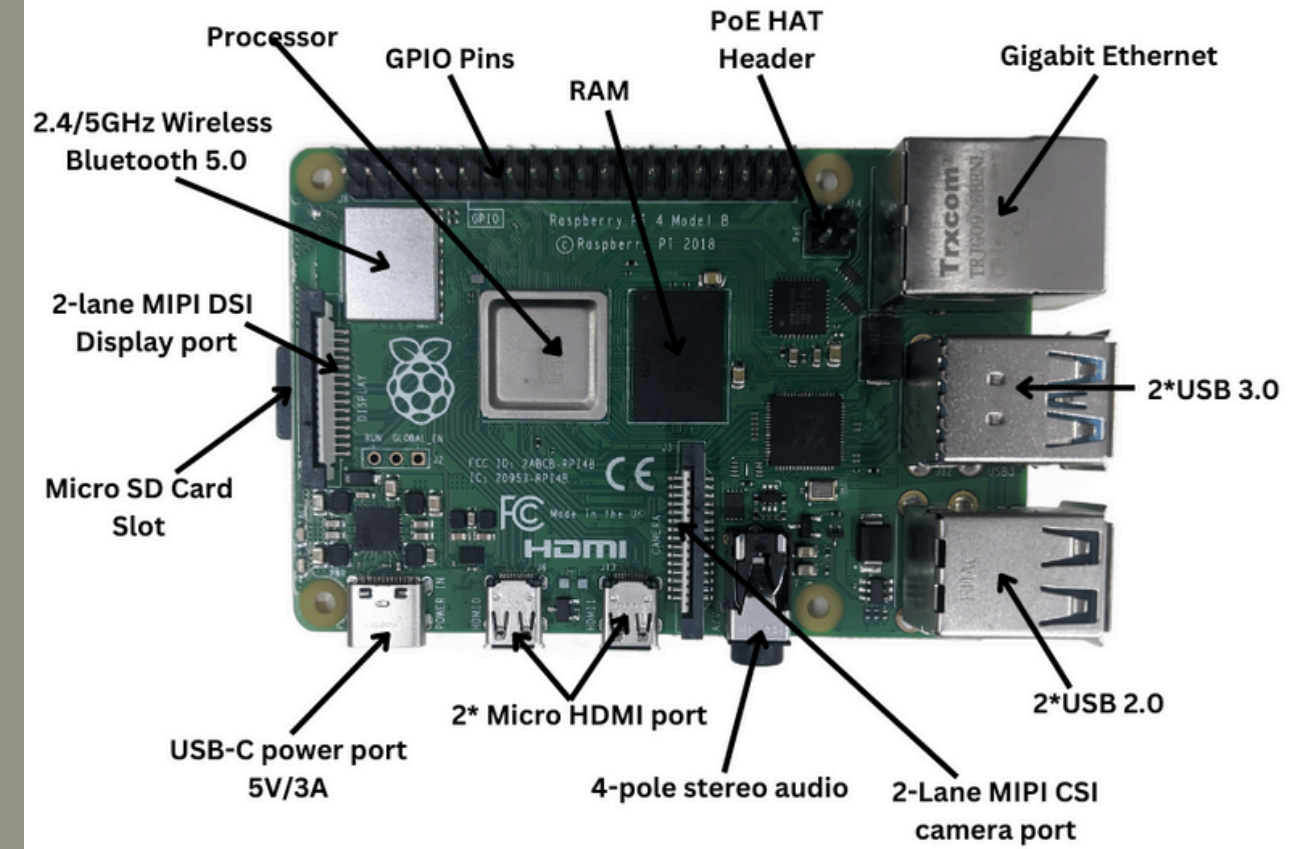


Hardware and Software Requirements

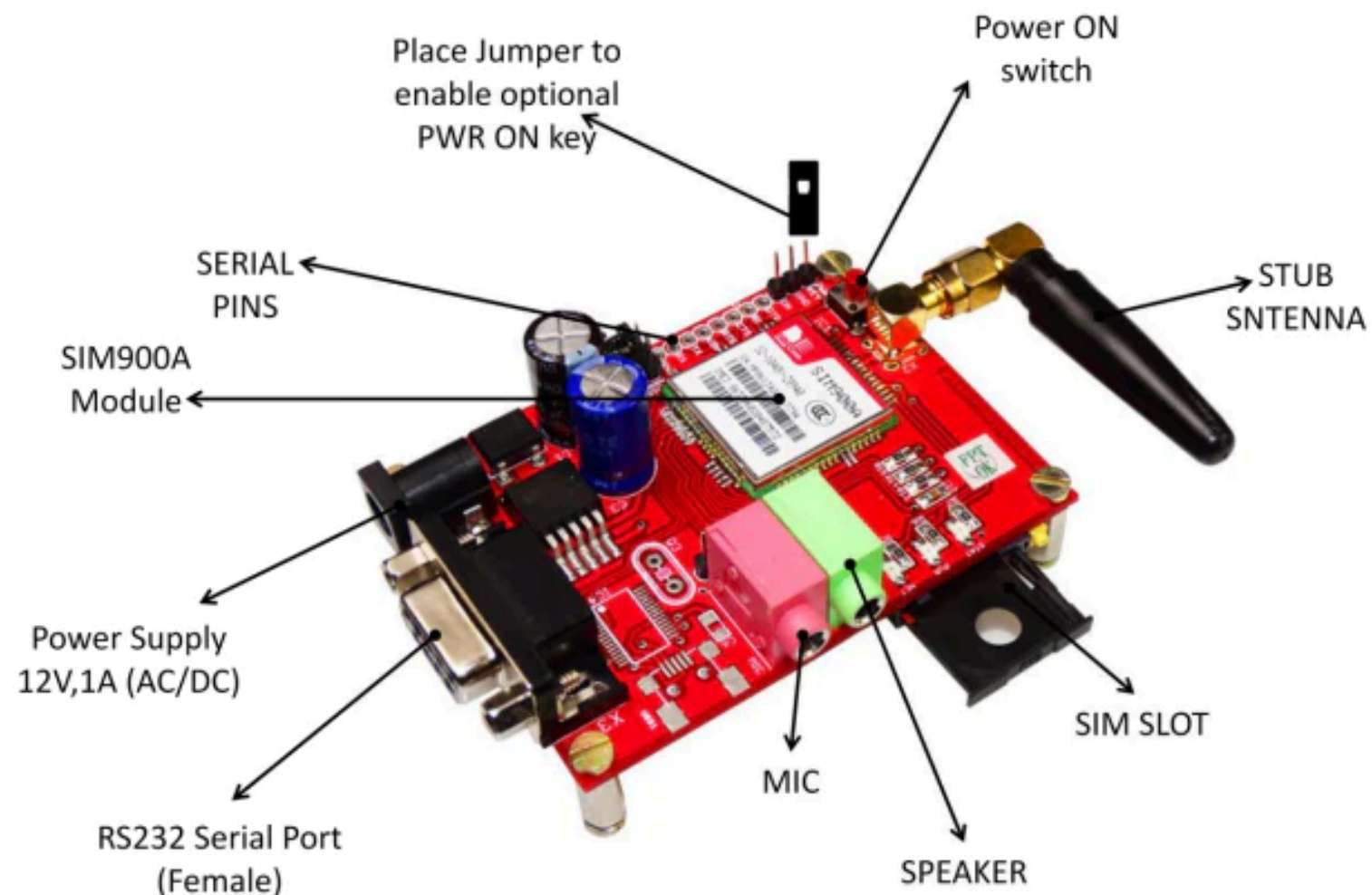
- ESP32 microcontroller boards
- Raspberry Pi (any model with WiFi and Bluetooth capabilities)
- Arduino Uno or similar microcontroller
- Bluetooth and WiFi antennas
- Breadboards, jumper wires, and other electronic components
- Python and Arduino IDE for programming
- GSM Module



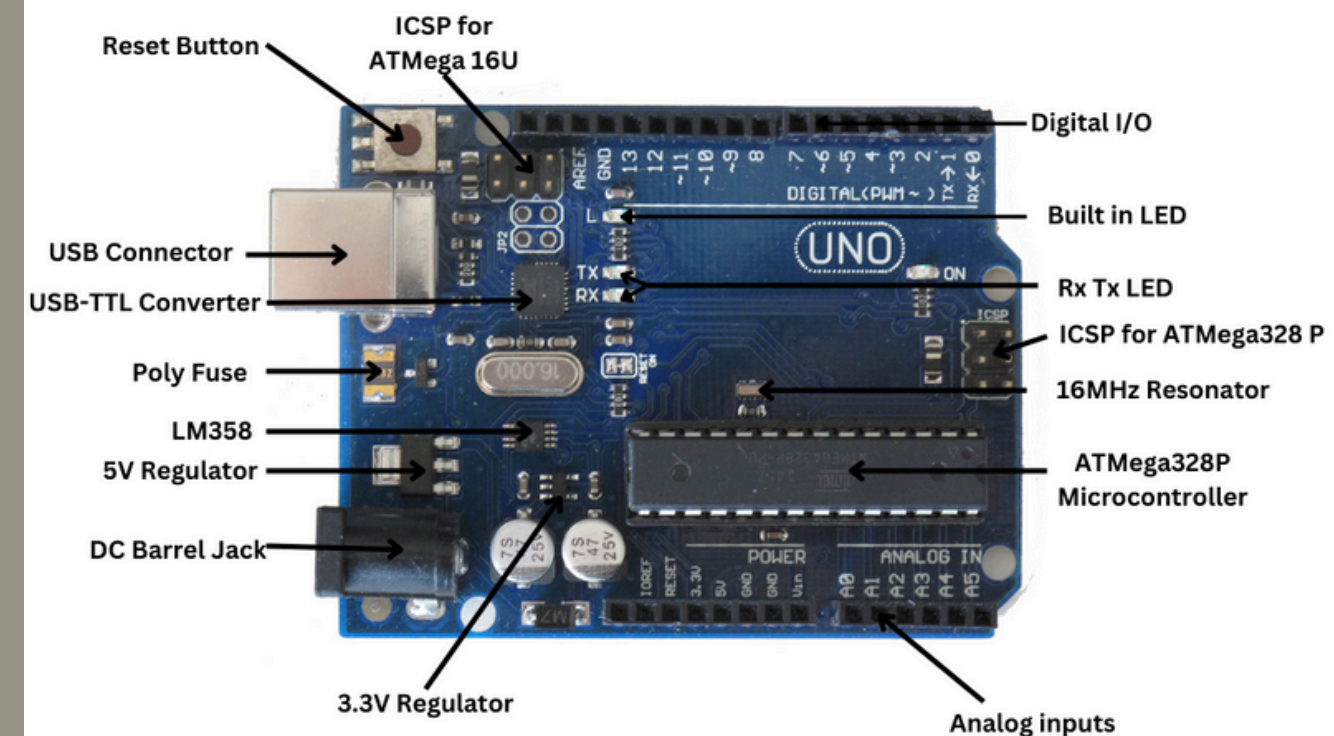
ESP-32



Raspberry Pi



GSM Module



Arduino UNO

ADVANTAGES

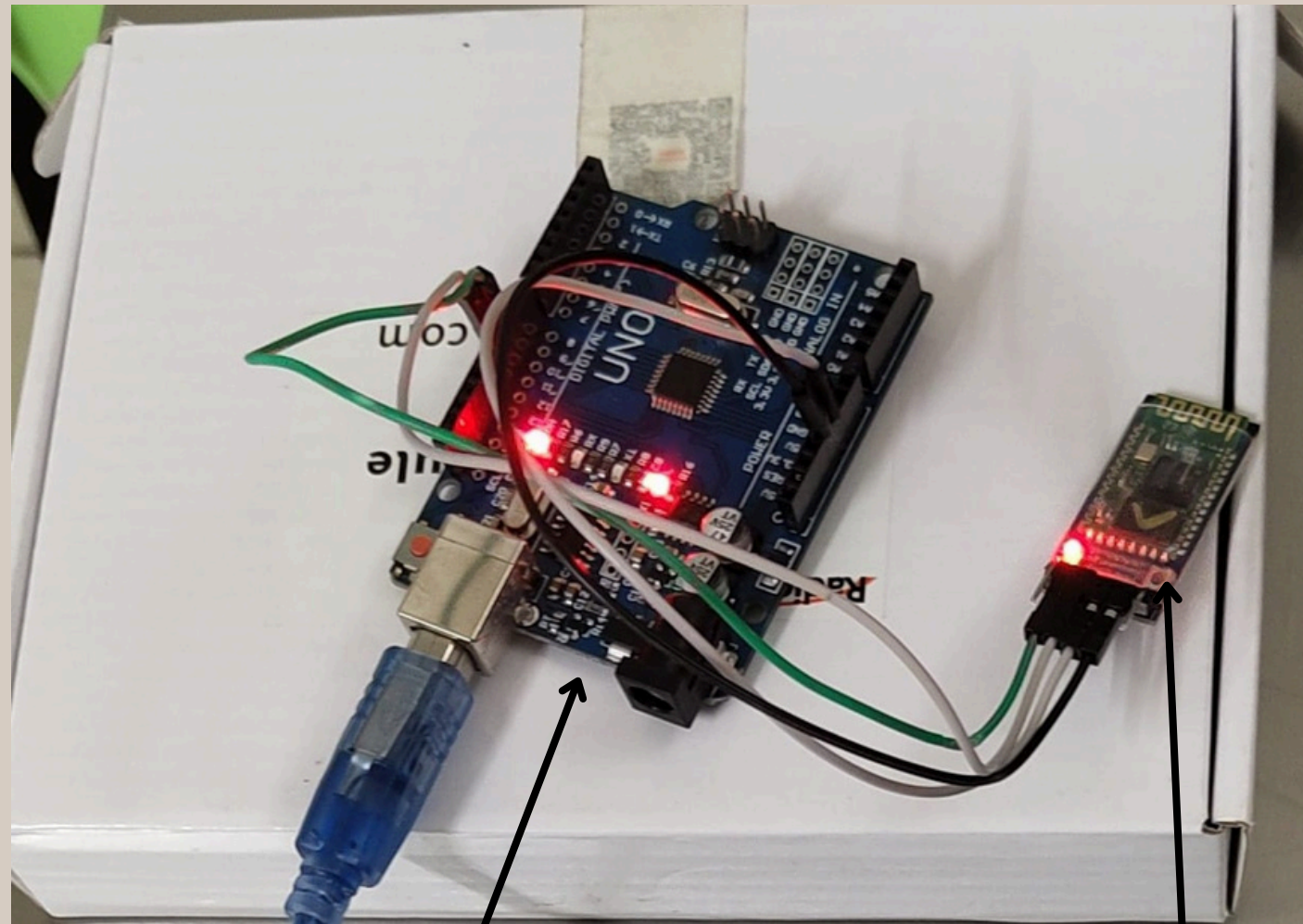
This project shows how we can use ESP32, Raspberry Pi, Arduino, and GSM devices to monitor traffic on highways without spending a lot of money. We use Bluetooth and WiFi sensors to gather data quickly and accurately. This helps us understand traffic conditions in real-time, making travel more predictable and allowing for better traffic management plans.

Methodology

1. **Hardware Setup:** Connect ESP32 to the Raspberry Pi. Ensure all devices are powered and connected for data communication.
2. **ESP32 Data Collection:** Use the ESP32 devices to scan for Bluetooth and WiFi signals from passing vehicles. Record the signal strength, MAC address, and other relevant information.
3. **Raspberry Pi Data Processing:** Receive and process the data from Raspberry Pi. Use Python scripts to analyze the data and extract traffic-related information such as MAC addresses, timestamps, etc.
4. **Data Storage and Analysis:** Store the processed data in a database for further analysis. Use tools like MongoDB and matplotlib in Python to analyze the data and generate reports on travel time reliability and traffic patterns.

Our Product

Arduino + hc 05

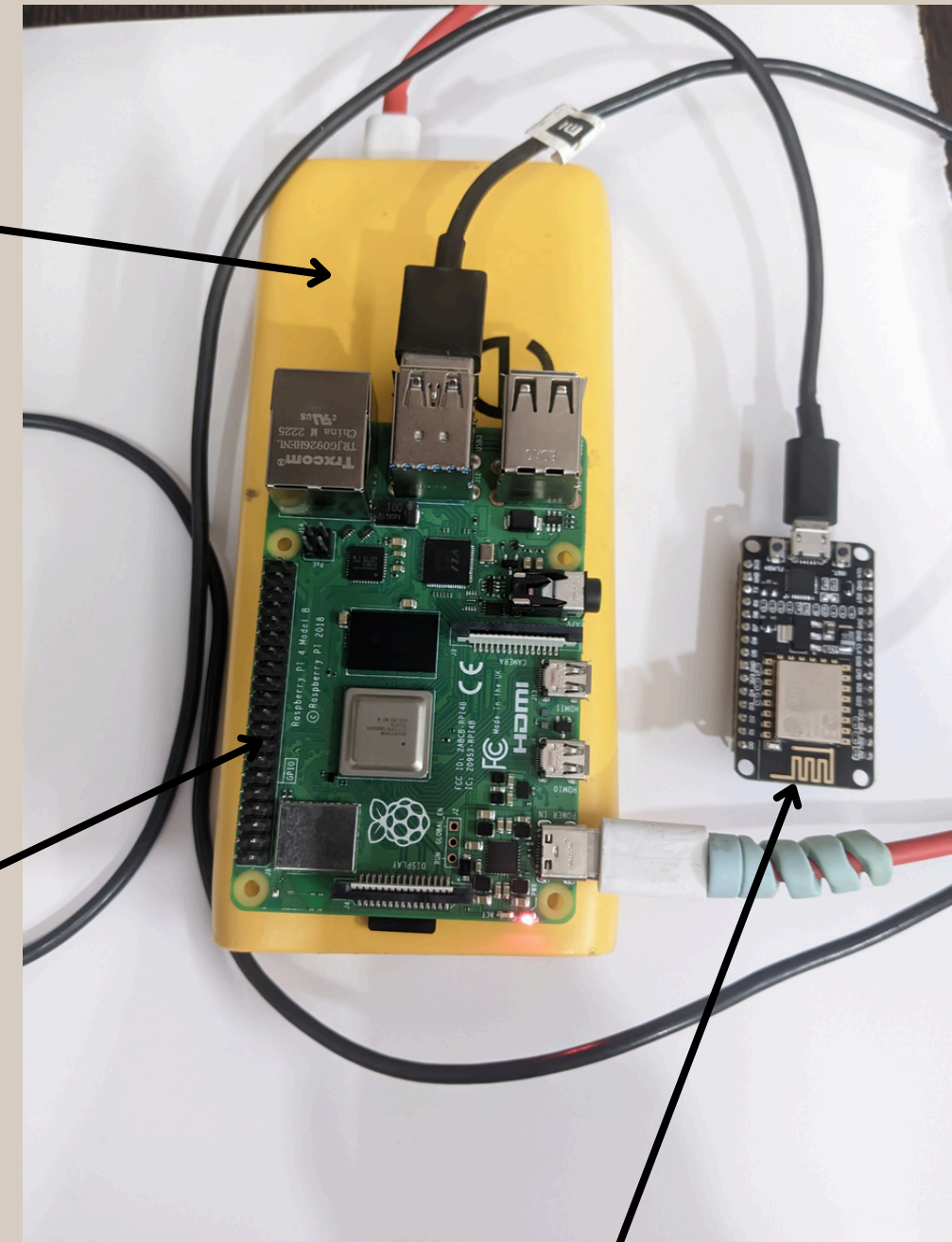


Arduino Uno

HC 05

Raspberry pi 4

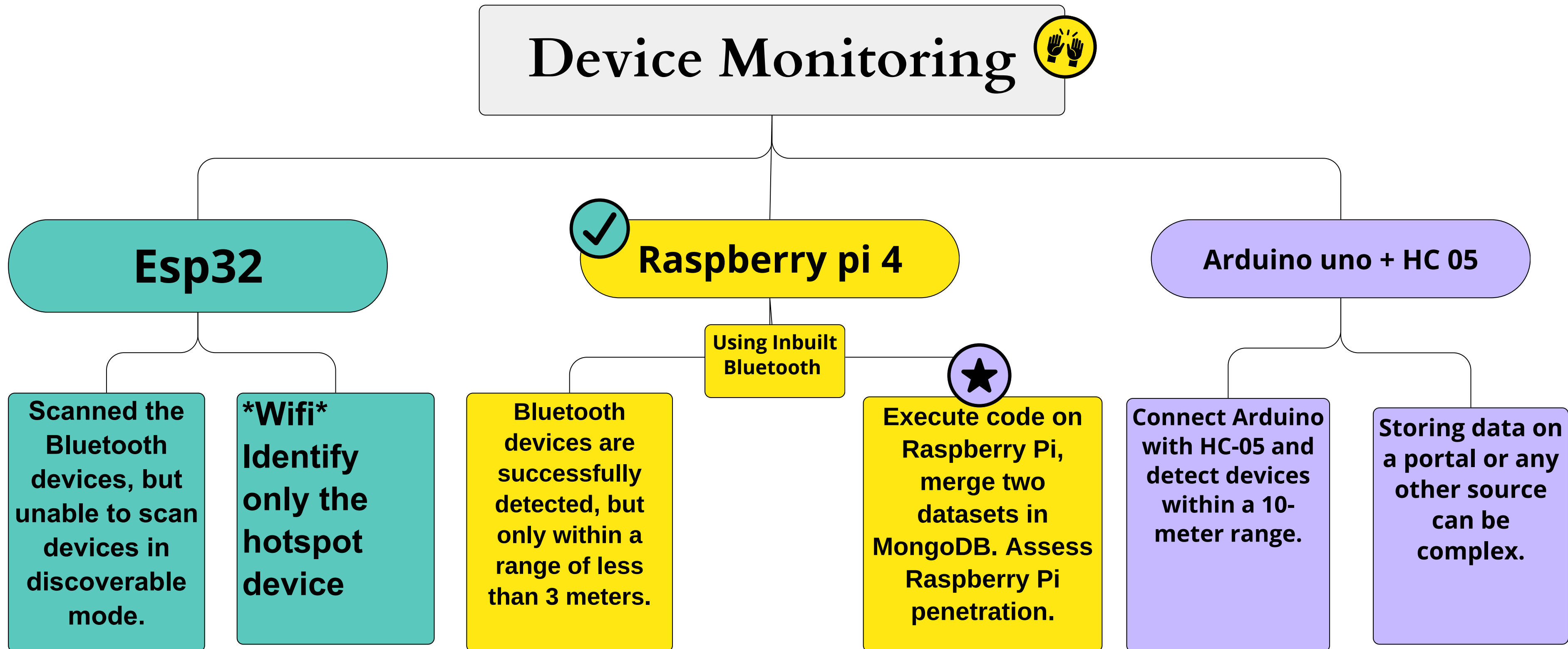
Power bank



Raspberry Pi

ESP 32

“Travel time reliability on highway using Bluetooth/ WiFi sensors”



Result Preview (Wifi)

wifi_scanner.py - Visual Studio Code

File Edit Selection View Go Run Terminal Help

REFERENCES
No results.

wifi_scanner.py

```
home > Harshad > Desktop > wifi_scanner.py > ...
6  fi():
10 s = re.findall(r"Cell \d+ - Address: (\S+).?*ESSID:\"(\S*?)\".*?Last beacon: (\d+)", output, re.DOTALL)
11 [(mac, ssid if ssid else 'Hidden', int(timestamp)) for mac, ssid, timestamp in networks]
12
13 == "__main__":
14 vices = set()
15 ue:
16 i_networks = scan_wifi()
17 rent_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
18 mac, ssid, ts in wifi_networks:
19 if mac not in seen_devices:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python Debug Console

```
Harshad@Harshad:~/Desktop $ /usr/bin/env /bin/python /home/Harshad/.vscode/extensions/ms-python.python-2024.2.1/pythonFiles/lib/python/debugpy/adapter/../../debugpy/launcher 38263 -- /home/Harshad/Desktop/wifi_scanner.py
MAC: EE:91:5E:41:37:58, SSID: Kalyan, Timestamp: 2024-03-16 23:31:25
MAC: 72:A4:E0:BF:69:F7, SSID: Harshad, Timestamp: 2024-03-16 23:31:25
MAC: 88:B1:E1:28:7F:40, SSID: IITJMU_Guest, Timestamp: 2024-03-16 23:31:25
MAC: 96:B8:8B:25:12:9D, SSID: IIT_JMU, Timestamp: 2024-03-16 23:31:25
MAC: 88:B1:E1:28:86:32, SSID: IIT_JAMMU, Timestamp: 2024-03-16 23:31:25
MAC: 88:B1:E1:28:86:30, SSID: IITJMU_Guest, Timestamp: 2024-03-16 23:31:25
MAC: 88:B1:E1:28:86:31, SSID: IIT_JMU, Timestamp: 2024-03-16 23:31:25
MAC: 88:B1:E1:28:7F:42, SSID: IIT_JAMMU, Timestamp: 2024-03-16 23:31:25
MAC: 88:B1:E1:28:80:62, SSID: IIT_JAMMU, Timestamp: 2024-03-16 23:31:25
MAC: 88:B1:E1:28:74:B0, SSID: IITJMU_Guest, Timestamp: 2024-03-16 23:31:25
MAC: 88:B1:E1:28:74:B1, SSID: IIT_JMU, Timestamp: 2024-03-16 23:31:25
MAC: 88:B1:E1:28:74:B2, SSID: IIT_JAMMU, Timestamp: 2024-03-16 23:31:25
MAC: 88:B1:E1:28:80:60, SSID: IITJMU_Guest, Timestamp: 2024-03-16 23:32:03
MAC: 88:B1:E1:28:80:61, SSID: IIT_JMU, Timestamp: 2024-03-16 23:32:03
```


Result Preview (Bluetooth)

The screenshot displays the Arduino IDE 2.2.1 interface. The top toolbar includes icons for checking, running, and uploading code, along with a dropdown menu for the selected board, 'ESP32 Wrover Kit (all versions)'. The left sidebar shows the 'SKETCHBOOK' with a file named 'sketch_jan17a'. The main editor window shows the code for 'sketch_jan17a.ino', which contains a standard Arduino setup and loop structure. The bottom panel is the 'Serial Monitor', which is set to 'Both NL & CR' and '9600 baud'. It displays a Bluetooth scan result from an 'ESP32 Wrover Kit (all versions)' on the serial port '/dev/cu.usbserial-0001'. The scan results show a list of discovered devices with their names, addresses, manufacturer data, and RSSI values. The first device is 'FT_38083_D90F' with address 'b1:86:64:41:d9:0f' and RSSI -94. The second device is 'Device 0' with address '0a:1b:76:5d:a1:45' and RSSI -90. The third device is 'Device 1' with address '22:c1:8c:cb:36:16' and RSSI -79. The fourth device is 'Device 2' with address '31:93:23:3d:21:59' and RSSI -93. The fifth device is 'Device 3' with address '36:2e:79:30:e6:96' and RSSI -91.

```
void setup() {  
  // put your setup code here, to run once:  
}  
  
void loop() {  
  // put your main code here, to run repeatedly:  
}
```

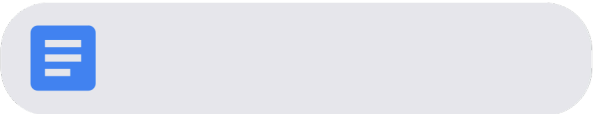
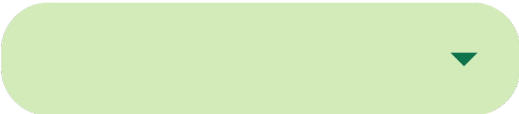
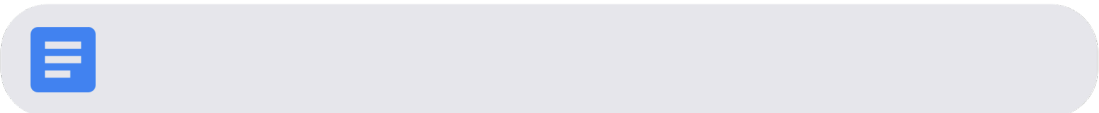
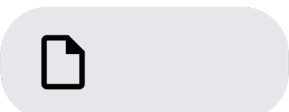
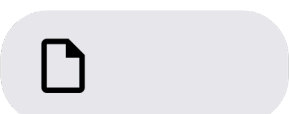
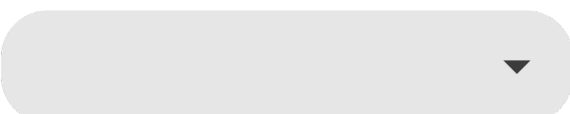
Serial Monitor X

Message (Enter to send message to 'ESP32 Wrover Kit (all versions)' on '/dev/cu.usbserial-0001')

Both NL & CR 9600 baud

19:55:57.030 -> Advertised Device: Name: , Address: 40:51:44:ca:30:72, manufacturer data: 4c0010073c11252360d438, txPower: 12, rssi: -93
19:55:57.178 -> Advertised Device: Name: , Address: 5b:a1:dc:17:64:39, manufacturer data: 4c00160800a608f9a31d8a41, rssi: -95
19:55:57.274 -> Advertised Device: Name: , Address: ef:98:25:ff:c5:ac, manufacturer data: 4c0012020001, rssi: -97
19:55:57.402 -> Advertised Device: Name: , Address: 7e:ea:af:ff:6d:ac, manufacturer data: 4c0010063f1ed4eed286, txPower: 12, rssi: -94
19:55:57.499 -> Advertised Device: Name: , Address: 74:26:42:d2:59:db, manufacturer data: 4c0010054f1c257cc0, txPower: 5, rssi: -92
19:55:57.627 -> Advertised Device: Name: , Address: 31:93:23:3d:21:59, manufacturer data: 4c00090603cbc0a82823, rssi: -93
19:55:57.755 -> Advertised Device: Name: , Address: e1:c9:06:a0:2c:88, manufacturer data: 4c0012020000, rssi: -96
19:55:57.851 -> Advertised Device: Name: , Address: 4e:fe:76:81:95:4f, manufacturer data: 4c00160800413671468677f6, rssi: -95
19:55:57.947 -> Advertised Device: Name: , Address: 50:79:8a:e0:0f:6d, manufacturer data: 4c001006221ec0eec5a9, txPower: 12, rssi: -91
19:55:58.075 -> Advertised Device: Name: , Address: 67:c5:6c:cb:81:12, manufacturer data: 4c0010053d1c7461d5, txPower: 12, rssi: -94
19:55:58.203 -> Advertised Device: Name: , Address: 6a:a4:df:cf:eb:1b, manufacturer data: 4c0010051c1865a16c, txPower: 12, rssi: -96
19:55:58.331 -> Advertised Device: Name: , Address: 6e:78:4d:9a:1d:08, manufacturer data: 4c001608004bf5b877c117d9, rssi: -80
19:55:58.427 -> Advertised Device: Name: , Address: cb:e6:fd:7b:d1:b3, manufacturer data: 4c0012020002, rssi: -78
19:55:58.555 -> Advertised Device: Name: , Address: d3:76:d9:19:ac:d2, manufacturer data: 4c0012020001, rssi: -57
19:55:58.651 -> Advertised Device: Name: , Address: 73:7b:99:fd:80:30, serviceUUID: 0000fef3-0000-1000-8000-00805f9b34fb, rssi: -97
19:55:58.780 -> Advertised Device: Name: FT_38083_D90F, Address: b1:86:64:41:d9:0f, manufacturer data: 4c42b1866441d90f56302e36382d38323031, rssi: -94
19:55:58.909 -> Advertised Device: Name: , Address: 3b:29:98:67:e7:60, manufacturer data: 0600010f2006f5727dd9111d5256fc8a5e76ccbae9f591619cfc77d59d,
19:55:59.069 -> Advertised Device: Name: , Address: c6:6c:67:55:c1:8a, manufacturer data: 4c0012020001, rssi: -90
19:55:59.525 -> Advertised Device: Name: , Address: 71:20:30:bb:16:ed, manufacturer data: 4c001608008853e5911d78a7, rssi: -97
19:56:00.115 -> Found 33 devices
19:56:00.147 -> Device 0: Name: , Address: 0a:1b:76:5d:a1:45, manufacturer data: 0600010f2022c3ae35a53b1d67da85d938458ed0e9c2184055bf7fc3af, rssi: -90
19:56:00.276 -> Device 1: Name: , Address: 22:c1:8c:cb:36:16, manufacturer data: 06000109212a91107675a4f44d5349, rssi: -79
19:56:00.405 -> Device 2: Name: , Address: 31:93:23:3d:21:59, manufacturer data: 4c00090603cbc0a82823, rssi: -93
19:56:00.501 -> Device 3: Name: , Address: 36:2e:79:30:e6:96, manufacturer data: 060001092022d05b14ad54b82a62867e3c4144945bd67eca9a70647e3f, rssi: -91

Progress:

File	Description	
	Wifi and Bluetooth detection using ESP32 <u>Bluetooth Device</u>	
	Bluetooth detection using Raspberry Pi	
	Bluetooth detection of moving vehicles	
	Use of external bluetooth and wifi module	

References:

Bluetooth Code By Esp32

Wi-Fi Code by Esp32

Bluetooth/Wi-fi Code By Raspberry Pi

Data Collection File

Thank you